

Reporte Proyecto

Estudio de Redes Neuronales Ortogonales

Resumen

En el presente trabajo se hace una revisión de una aproximación de las Redes Neuronales Ortogonales, en función de esto primero explicamos desde el punto de vista geométrico y desde la teoría del aprendizaje automático el funcionamiento de una Red Neuronal Profunda. Para esta explicación y para los experimentos posteriores seleccionamos el conocido problema de clasificación de dígitos utilizando el dataset de MNIST. Posteriormente implementamos la aproximación de una Red neuronal ortogonal utilizando el algoritmo de regularización ortogonal; analizamos los resultados de estabilidad numérica en el entrenamiento, tiempos de entrenamiento y de generalización entre cinco modelos entrenados con distintos valores de penalización. Los resultados arrojan que la selección del hiperparámetro del valor de regularización es vital para obtener un modelo con un balance adecuado entre generalización y estabilidad numérica en el entrenamiento.

1. Introducción

Las Redes Neuronales Profundas como agentes de entrenamiento automático han demostrado ser bastante útiles para modelar funciones y relaciones complejas. Sin embargo, el entrenamiento efectivo de estas arquitecturas, especialmente a medida que aumenta su profundidad, presenta desafíos numéricos y de optimización no triviales. En la literatura se discute ampliamente cómo problemas como la explosión y el desvanecimiento del gradiente pueden dificultar la convergencia del modelo, mientras que el sobreajuste (overfitting) limita su capacidad para generalizar ante datos no vistos[4].

En este contexto, la imposición de restricciones de ortogonalidad sobre las matrices de pesos surge como una solución elegante y geoméricamente fundamentada. Las Redes Neuronales Profundas Ortogonales (OrthDNNs) buscan garantizar que las transformaciones lineales entre capas preserven la norma de los vectores (isometría) y los ángulos entre ellos. Como señalan Li et al. [3], esta propiedad no solo mitiga los problemas de estabilidad numérica asegurando un flujo de gradiente más estable durante la retropropagación, sino que también favorece la generalización al decorrelacionar las características aprendidas por el modelo.

En este reporte abordamos el estudio y la implementación de las OrthDNNs conectando herramientas del Álgebra Lineal Numérica, específicamente las transformaciones de Householder y la descomposición QR, con la optimización de redes neuronales. A diferencia de un enfoque puramente teórico, centramos

nuestro trabajo en la experimentación práctica aplicada al problema de clasificación de imágenes utilizando el dataset MNIST.

El objetivo principal es evaluar el impacto de la regularización ortogonal en el proceso de aprendizaje. Para ello, implementamos una arquitectura base y comparamos su desempeño bajo distintos coeficientes de penalización (λ). A través de estos experimentos, analizamos no solo la precisión (accuracy) y la función de pérdida, sino también métricas críticas como el error de ortogonalidad por capa y el número de condición de las matrices de pesos. De esta forma, buscamos determinar empíricamente si la teoría se traduce en una ventaja práctica y cuál es el balance costo-beneficio entre la estabilidad numérica y el tiempo de cómputo adicional que estas restricciones imponen.

2. Preliminares

En esta sección se introducen los conceptos necesarios para estudiar las *redes neuronales profundas ortogonales*. Primero se describen las redes neuronales superficiales y profundas, luego se plantea el problema de clasificación de dígitos (MNIST) que se utiliza como caso de estudio, y finalmente se motiva el uso de restricciones de ortogonalidad en los pesos.

2.1 Redes Neuronales

2.1.1 Redes Neuronales Superficiales

Una *red neuronal superficial* (perceptrón multicapa con una capa oculta) puede modelarse como una función no lineal parametrizada por pesos y sesgos. Para una salida escalar, una forma típica es

$$F(x) = \phi_0 + \sum_{i=1}^n \phi_i a \left(\sum_{j=1}^d w_{ij} x_j + b_i \right), \quad (1)$$

donde $x \in \mathbb{R}^d$ es la entrada, n es el número de unidades ocultas, a es una función de activación y $\{w_{ij}\}$, $\{b_i\}$, $\{\phi_i\}$ son los parámetros del modelo. Para fijar ideas, si $d = 3$ y $n = 3$, las activaciones ocultas pueden escribirse como

$$h_i := a(w_{i1}x_1 + w_{i2}x_2 + w_{i3}x_3 + b_i), \quad i = 1, 2, 3.$$

Si el modelo produce dos salidas, se usan pesos de salida distintos para cada componente. En notación vectorial,

$$y = \Phi_0 + \sum_{i=1}^3 \Phi_i h_i, \quad \Phi_i \in \mathbb{R}^2, \quad (2)$$

donde $\Phi_i = \begin{bmatrix} \phi_{1,i} \\ \phi_{2,i} \end{bmatrix}$ recoge los pesos que conectan la i -ésima neurona oculta con las dos salidas.

Este tipo de modelo se representa de forma conveniente mediante un grafo computacional; en la Figura 1 se muestra el correspondiente a la ecuación (2) (adaptado de [4]).

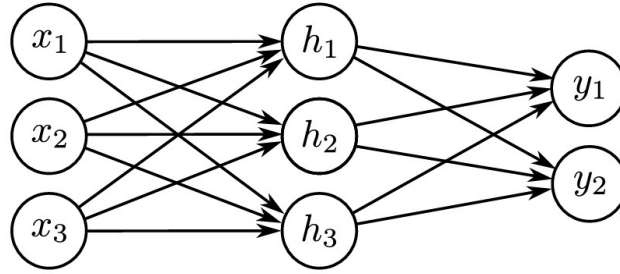


Figura 1: Grafo computacional de una red superficial con tres unidades ocultas y dos salidas. Adaptado de [4].

2.1.2 Redes Neuronales Profundas

Una *red neuronal profunda* se obtiene al componer varias transformaciones lineales seguidas de no linealidades. Sea $d_0 = d$ la dimensión de entrada y d_i el número de neuronas en la i -ésima capa; entonces los pesos y sesgos satisfacen $W_i \in \mathbb{R}^{d_i \times d_{i-1}}$ y $b_i \in \mathbb{R}^{d_i}$. Para k capas, una parametrización estándar es

$$h_0 = x, \quad h_i = a_i(W_i h_{i-1} + b_i) \quad (i = 1, \dots, k-1), \quad F(x) = W_k h_{k-1} + b_k. \quad (3)$$

Retomando el ejemplo anterior, si se agregan dos capas ocultas adicionales, el grafo computacional puede representarse como en la Figura 2.

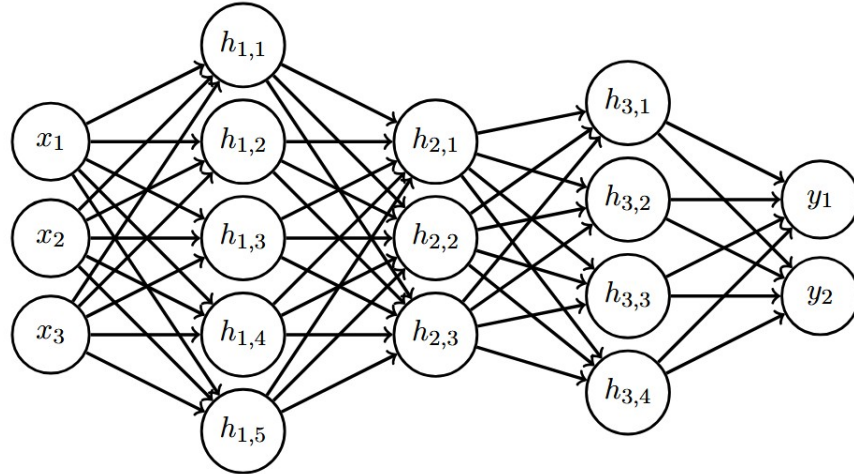


Figura 2: Grafo computacional ilustrativo para una red profunda (ver ecuación (4)).

Para un caso concreto con dimensiones compatibles (salida en $\mathbb{R}^2$), puede escribirse

$$y = W_4 h_3 + b_4, \quad (4)$$

con $W_4 \in \mathbb{R}^{2 \times d_3}$ y $b_4 \in \mathbb{R}^2$.

En aprendizaje automático se eligen los parámetros $\theta = \{W_i, b_i\}$ minimizando una función de pérdida $\mathcal{L}$ sobre un conjunto de entrenamiento, de modo que el modelo generalice a muestras no vistas.

2.2 Problema de Clasificación de Dígitos

Como caso de estudio se considera la clasificación de dígitos manuscritos del conjunto MNIST [2]. Cada imagen es de 28×28 píxeles (784 entradas) y el objetivo es predecir una de diez clases (0–9). Una arquitectura base consiste en una entrada de 784 características, varias capas ocultas y una capa final con 10 salidas.

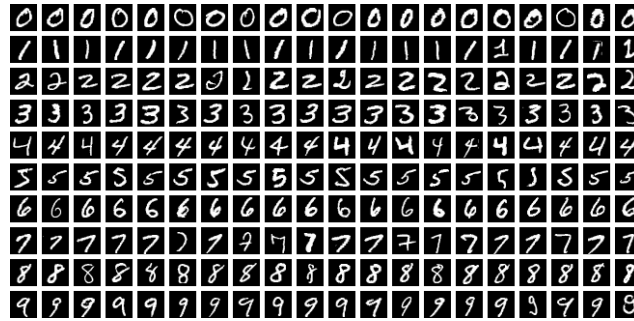


Figura 3: Ejemplos del conjunto MNIST para clasificación de dígitos manuscritos [2].

El grafo computacional de una propuesta de red totalmente conectada para este problema se ilustra en la Figura 4.

En este modelo, la primera capa recibe los píxeles de la imagen, las capas ocultas extraen representaciones intermedias y la última capa produce las puntuaciones (o probabilidades, tras una *softmax*) asociadas a cada dígito.¹

Intuitivamente, imágenes similares tienden a inducir activaciones similares a lo largo de la red. Preservar dichas relaciones a través de las capas motiva el estudio de arquitecturas con restricciones sobre los pesos, como es el caso de las redes profundas ortogonales.

2.3 Redes Neuronales Profundas Ortogonales

Una *red neuronal profunda ortogonal* es una red profunda en la que (algunas o todas) las matrices de pesos W_i se restringen a ser ortogonales (o aproximadamente ortogonales) [3]. En el caso cuadrado, la condición $W^\top W = I$ implica preservación de norma,

$$\|Wx\|_2^2 = (Wx)^\top (Wx) = x^\top W^\top W x = \|x\|_2^2. \quad (5)$$

¹Una visualización interactiva de un perceptrón multicapa puede consultarse en https://adamharley.com/nn_vis/mlp/2d.html.

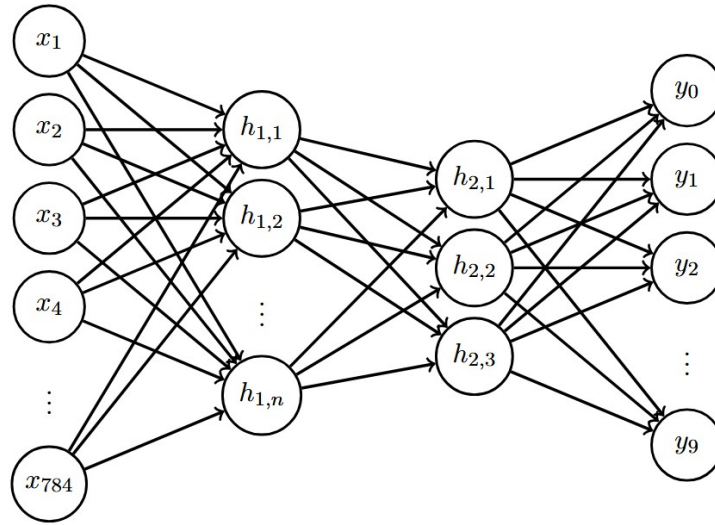


Figura 4: Grafo computacional de un perceptrón multicapa para clasificación de imágenes.

En una red profunda, la propagación hacia adelante y hacia atrás involucra productos sucesivos de matrices (y de sus transpuestas). Sin restricciones, pequeñas desviaciones en los valores singulares de los W_i pueden amplificarse con la profundidad, contribuyendo a inestabilidades numéricas y a gradientes que se desvanecen o explotan [1]. En contraste, imponer (aproximadamente) ortogonalidad ayuda a estabilizar la magnitud de las activaciones y de los gradientes a lo largo de las capas.

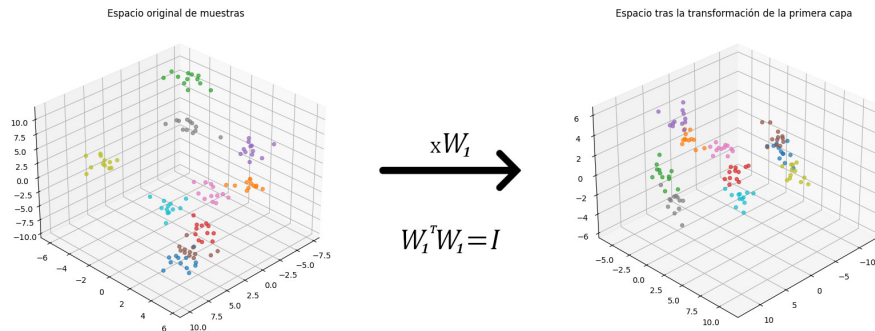


Figura 5: Ilustración de preservación de distancias y ángulos entre muestras bajo una transformación ortogonal.

2.4 Entrenamiento de los Modelos

El entrenamiento se plantea como un problema de optimización: dada una función de pérdida $\mathcal{L} = \mathcal{L}(F_\theta(x), y)$, se busca

$$\min_{\theta} \mathbb{E}[\mathcal{L}],$$

lo cual en la práctica se aborda con variantes de descenso por gradiente estocástico. En su forma básica, los parámetros se actualizan como

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial \mathcal{L}}{\partial \theta}(\theta_t), \quad (6)$$

donde $\alpha > 0$ es la tasa de aprendizaje. Durante este proceso pueden aparecer (i) inestabilidades en el flujo del gradiente a través de capas profundas, y (ii) sobreajuste, cuando el modelo se adapta excesivamente al conjunto de entrenamiento y generaliza mal. La ortogonalidad (o penalizaciones que la promueven) puede verse como un mecanismo de regularización estructural sobre los pesos, complementario a otras técnicas de regularización.

3. Metodología: Regularización Ortogonal

Como se discutió en la sección anterior, imponer restricciones estrictas de ortogonalidad (Hard Orthogonality) mediante métodos como la descomposición QR o SVD en cada paso de entrenamiento resulta computacionalmente costoso e inviable para redes profundas. Por lo tanto, en este trabajo adoptamos una estrategia de *ortogonalidad suave* (Soft Orthogonality).

Esta aproximación consiste en relajar la restricción geométrica incorporándola directamente en la función de optimización. Modificamos la función de pérdida original $\mathcal{L}$ (en nuestro caso, la entropía cruzada para clasificación) añadiendo un término de penalización que castiga la desviación de las matrices de pesos respecto a la ortogonalidad.

Definimos el término de regularización ortogonal para una capa l con pesos W_l como:

$$\mathcal{R}(W_l) = \|W_l^\top W_l - I\|_F^2 \quad (7)$$

donde $\|\cdot\|_F$ denota la norma de Frobenius e I es la matriz identidad. De este modo, la función de costo global que optimizamos es:

$$\mathcal{L}_{total}(\theta) = \mathcal{L}(\theta) + \lambda \sum_{l=1}^K \mathcal{R}(W_l) \quad (8)$$

Aquí, $\lambda \in \mathbb{R}^+$ es un hiperparámetro de regularización que controla la fuerza con la que imponemos la ortogonalidad.

- Si $\lambda = 0$, el modelo se comporta como una red neuronal estándar sin restricciones.
- Si $\lambda \rightarrow \infty$, el modelo prioriza la ortogonalidad sobre la precisión de la clasificación, lo que puede llevar a un subajuste (underfitting).

El gradiente de este término de regularización respecto a los pesos es computacionalmente eficiente de calcular:

$$\frac{\partial \mathcal{R}}{\partial W} = 4W(W^\top W - I) \quad (9)$$

Esto permite actualizar los pesos utilizando descenso de gradiente estándar sin necesidad de proyecciones complejas sobre la variedad de Stiefel.

4. Diseño Experimental

Para evaluar el impacto de la regularización ortogonal en la estabilidad y el desempeño del modelo, diseñamos dos experimentos controlados utilizando el dataset MNIST. En el primero se implementó una arquitectura de Red Neuronal Profunda (DNN) con las siguientes características:

- **Entrada:** Vector de 784 dimensiones (imágenes aplanadas de 28×28).
- **Capas Ocultas:** Dos capas densas con activación ReLU.
- **Salida:** Capa de 10 neuronas (correspondientes a los dígitos 0-9) con activación Softmax.

El objetivo central del experimento es comparar el comportamiento del modelo bajo distintos valores del coeficiente de regularización λ . Se entrenaron cinco modelos independientes con los siguientes valores:

$$\lambda \in \{0, 0.01, 0.1, 0.5, 1.0\}$$

Todos los modelos fueron entrenados durante el mismo número de épocas, utilizando el mismo optimizador y tasa de aprendizaje para garantizar la comparabilidad de los resultados. Se monitorearon cuatro métricas principales: la función de pérdida (Loss), la precisión en validación (Accuracy), el error de ortogonalidad ($\|W^T W - I\|$) y el número de condición de las matrices de pesos.

En el segundo experimento tomamos el modelo que obtuvo mejor desempeño del experimento anterior y se comparó con un modelo no ortogonal, variando esta vez el número de capas ocultas:

$$n = \{6, 10, 14, 18, 22, 26\}$$

En este caso, todas las capas internas extras que se añaden por modelo tiene 128 neuronas. Se monitorean el tiempo de ejecución y el error de ortogonalización total de todas las capas de los modelos.

5. Resultados y Discusión

A continuación, presentamos los resultados obtenidos tras el proceso de entrenamiento.

5.1 Desempeño en Clasificación y Convergencia

En la Figura 6 observamos la evolución de la función de pérdida y la precisión durante el entrenamiento.

Se observa que los modelos con λ moderado (e.g., 0.01, 0.1) logran converger a una precisión comparable al modelo base ($\lambda = 0$). Sin embargo, cuando la penalización es muy alta ($\lambda = 1.0$), la restricción de ortogonalidad compite agresivamente con la tarea de clasificación, lo que puede ralentizar la convergencia o converger a una solución subóptima en términos de accuracy. Esto confirma que existe un compromiso entre imponer estructura geométrica y permitir libertad de representación.

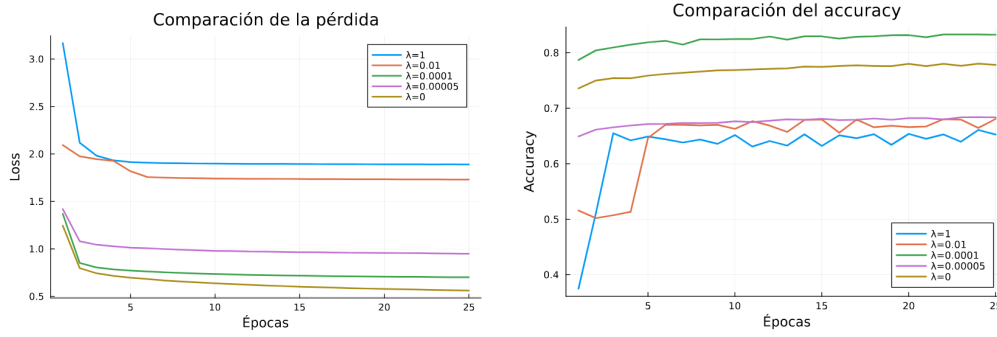


Figura 6: Comparación de la pérdida (izquierda) y la precisión (derecha) para distintos valores de λ

5.2 Estabilidad Numérica y Ortogonalidad

La efectividad de la regularización se evidencia al analizar el error de ortogonalidad y el número de condición, mostrados en la Figura 7 y la Figura 9, respectivamente.

El modelo sin regularización ($\lambda = 0$) muestra un crecimiento rápido en el error de ortogonalidad, lo que indica que sus matrices de pesos se alejan de ser isométricas. Por el contrario, los modelos regularizados mantienen este error acotado cerca de cero. Esto puede observarse mejor en los resultados obtenidos en la Figura 8 del segundo experimento, en donde se ve que mientras más grande es el modelo, en una DNN el error incrementa con un comportamiento lineal, mientras que el modelo ortogonal se mantiene constante.

Más importante aún es el análisis del *Número de Condición* ($\kappa(W)$). Un número de condición bajo (cercano a 1) indica que la matriz de pesos preserva bien la magnitud de los gradientes, evitando los problemas de desvanecimiento o explosión. Los resultados muestran que a medida que aumentamos λ , $\kappa(W)$ se mantiene significativamente más bajo y estable a lo largo de las épocas en comparación con el modelo base. Esto valida la hipótesis de que las OrthDNNs son numéricamente más estables.

5.3 Tiempo de Ejecución

Se ha mostrado que los modelos ortogonales son numéricamente más estables, sin embargo, es bien sabido que el cálculo de ortogonalización de matrices es costoso computacionalmente hablando. Esto se muestra en la Figura 10 y la Figura 11.

En los tiempos tomados el primer experimento se puede observar que, por época, los modelos ortogonales tardan un poco más del triple de lo que tarda un modelo sin restricción. Además, se observa que el modelo ortogonal tarda aproximadamente un poco más de el doble en ejecución total del entrenamiento a medida que el número de capas aumenta. Sin embargo, no podemos concluir que escale o que el orden de magnitud aumente; para redes neuronales con pocas capas ambos modelos parecen tener un comportamiento asintótico lineal. De esta forma podemos concluir un claro compromiso entre tiempo de ejecución y estabilidad numérica al seleccionar un modelo ortogonal.

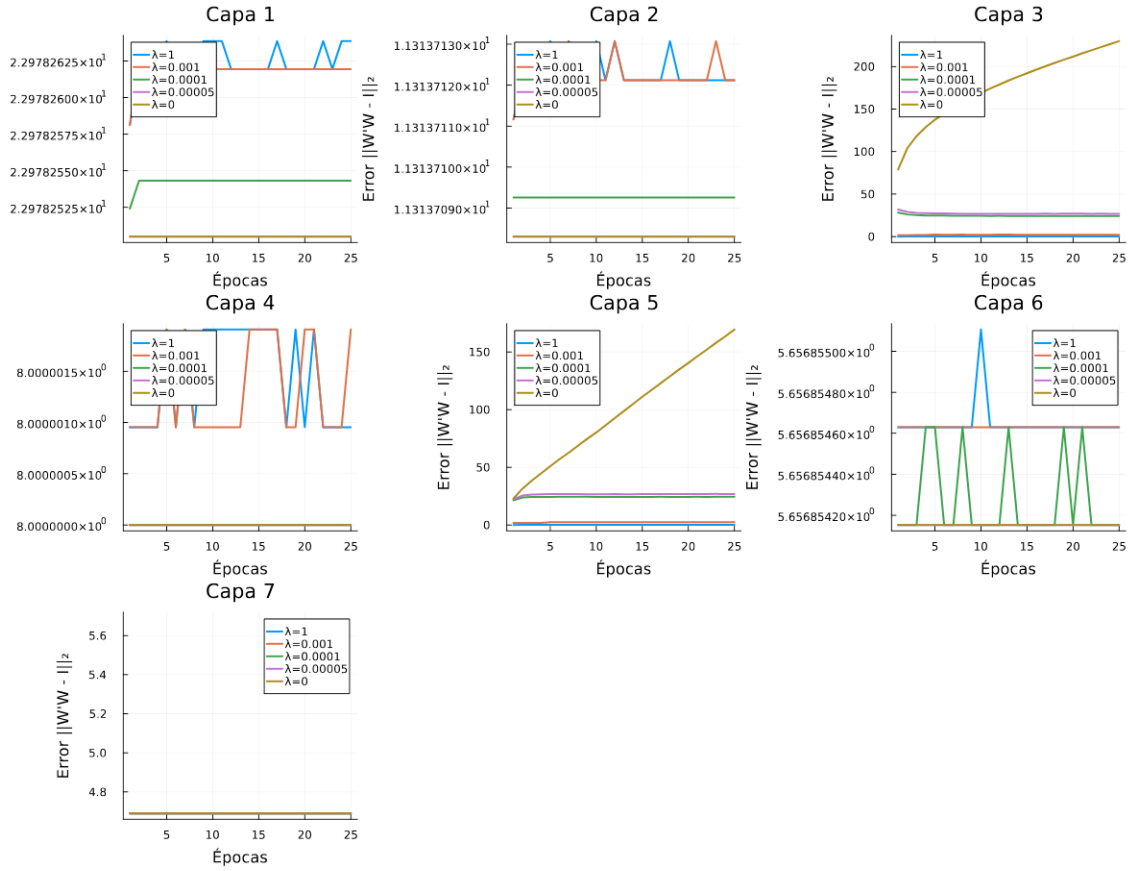


Figura 7: Error de ortogonalidad capa por capa en cada uno de los modelos.

6. Conclusiones

En este trabajo hemos explorado la implementación y el impacto de las Redes Neuronales Ortogonales aplicadas al problema de clasificación de MNIST. A través de la experimentación con regularización suave, hemos llegado a las siguientes conclusiones:

1. **Estabilidad Numérica:** La regularización ortogonal mejora drásticamente el condicionamiento de las matrices de pesos. Esto sugiere que las OrthDNNs son menos propensas a sufrir inestabilidades numéricas en redes profundas, facilitando la propagación de gradientes limpios.
2. **Trade-off entre Restricción y Desempeño:** Existe un delicado equilibrio al seleccionar el hiperparámetro λ . Un valor demasiado alto fuerza la ortogonalidad a expensas de la capacidad del modelo para minimizar el error de clasificación, mientras que un valor muy bajo no aporta los beneficios de estabilidad deseados. La selección de λ es, por tanto, vital.
3. **Generalización:** Aunque en una red poco profunda y un dataset sencillo como MNIST las diferencias en generalización pueden ser sutiles, la estructura impuesta por la ortogonalidad

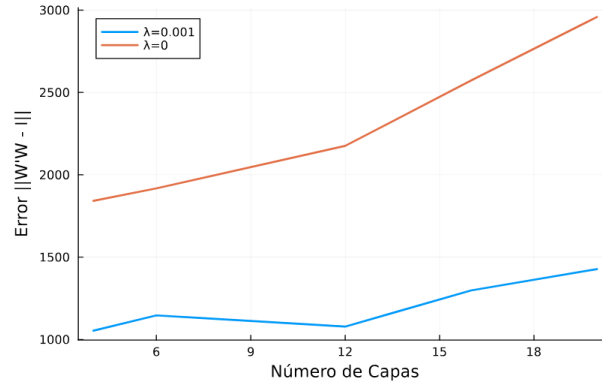


Figura 8: Comparativa del error total entre los modelos con $\lambda = 0$ y $\lambda = 0.0001$.

actúa como un regularizador efectivo, reduciendo la varianza de los pesos y potencialmente decorrelacionando las características aprendidas.

Como trabajo futuro, sería relevante extender este análisis a arquitecturas más profundas (como ResNets o RNNs) donde los problemas de gradiente son más críticos, y comparar la regularización suave con métodos de optimización en variedades de Stiefel para evaluar el costo computacional frente a la exactitud de la ortogonalidad.

Declaración de uso de IA

Este reporte ha sido elaborado con la asistencia de herramientas de Inteligencia Artificial. Específicamente, se utilizaron modelos de lenguaje para la corrección de estilo, sugerencias de estructura en $\text{\LaTeX}$ y la implementación de algunos códigos. Sin embargo, la ejecución de los experimentos, la recopilación de datos y el análisis crítico de los resultados presentados son autoría original del estudiante.

Referencias

- [1] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.
- [2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [3] S. Li, K. Jia, Y. Wen, T. Liu, and D. Tao. Orthogonal deep neural networks. *IEEE transactions on pattern analysis and machine intelligence*, 43(4):1352–1368, 2019.
- [4] S. J. D. Prince. *Understanding Deep Learning*. The MIT Press, 2023.

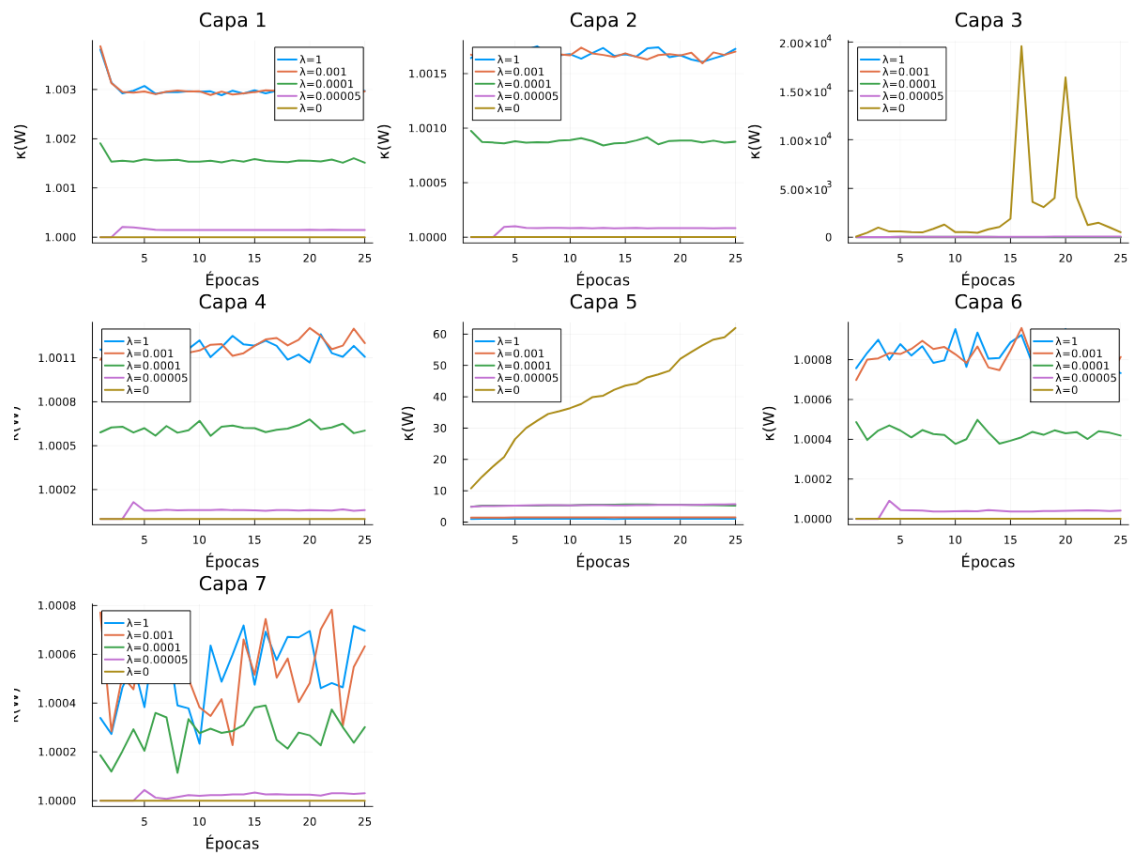


Figura 9: Número de condición capa por capa en cada uno de los modelos.

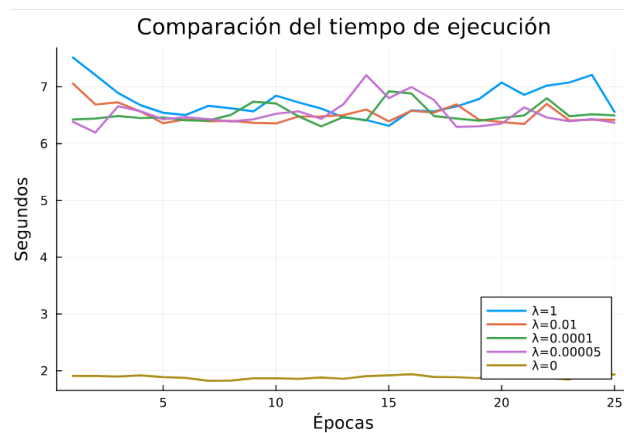


Figura 10: Tiempo de ejecución de cada época en el entrenamiento de los modelos en el primer experimento.

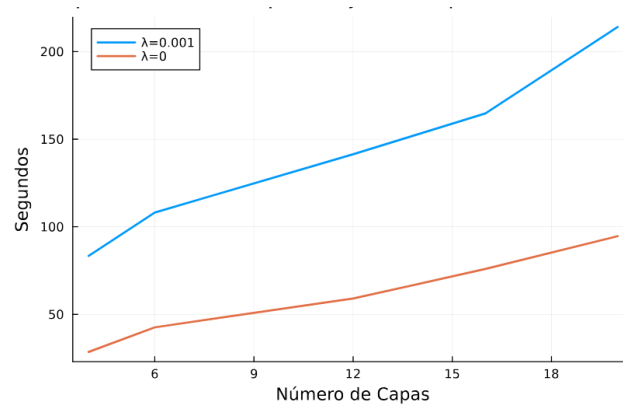


Figura 11: Tiempo de ejecución entre los modelos con $\lambda = 0$ y $\lambda = 0.0001$.